

UNIT - IV UNSUPERVISED LEARNING

Introduction to clustering, Hierarchical: AGNES, DIANA, Partitional: K-means clustering, K-Mode Clustering, Self-Organizing Map, Expectation Maximization(EM), Gaussian Mixture Models, Principal Component Analysis (PCA).

4.1 Introduction to clustering

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things. It can be defined as:

“Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.”

Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format**

Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.



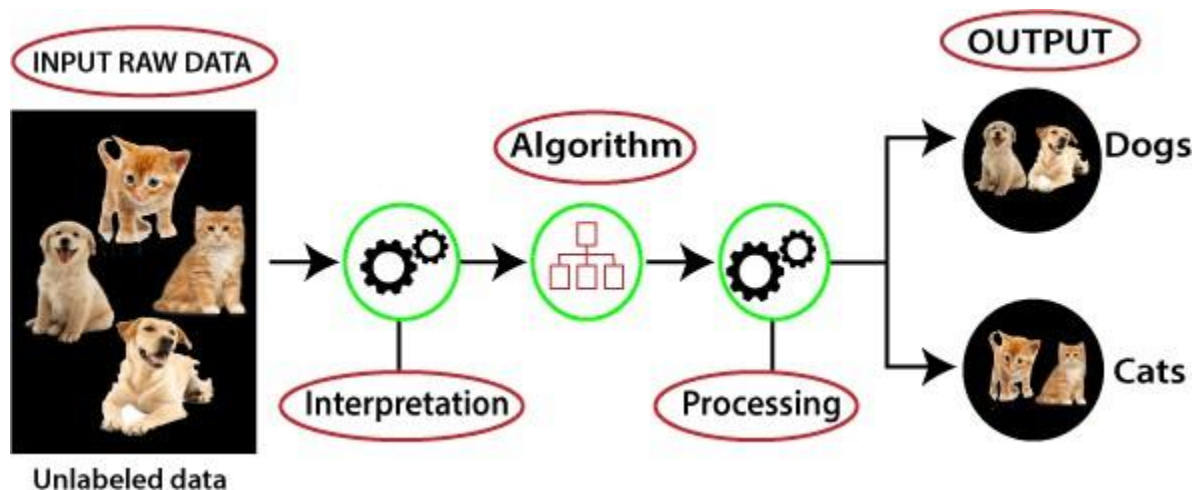
Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

Working of Unsupervised Learning

Working of unsupervised learning can be understood by the below diagram:

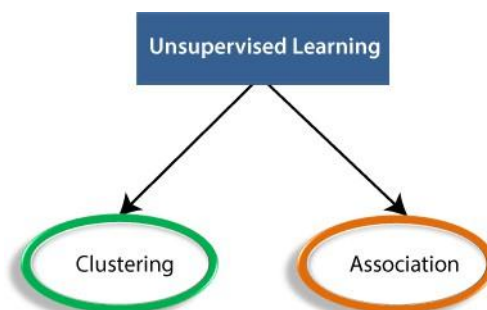


Here, we have taken an unlabeled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabeled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.

Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:



- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remain into a group and has less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.
- **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

Unsupervised Learning algorithms:

Below is the list of some popular unsupervised learning algorithms:

- K-means clustering
- KNN (k-nearest neighbors)
- Hierarchical clustering
- Anomaly detection
- Neural Networks
- Principle Component Analysis
- Independent Component Analysis
- Apriori algorithm
- Singular value decomposition

Advantages of Unsupervised Learning

- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
- Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled data.

Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.

Supervised Learning	Unsupervised Learning
Supervised learning algorithms are trained using labeled data.	Unsupervised learning algorithms are trained using unlabeled data.
Supervised learning model takes direct feedback to check if it is predicting correct output or not.	Unsupervised learning model does not take any feedback.
Supervised learning model predicts the output.	Unsupervised learning model finds the hidden patterns in data.
In supervised learning, input data is provided to the model along with the output.	In unsupervised learning, only input data is provided to the model.
The goal of supervised learning is to train the model so that it can predict the output when it is given new data.	The goal of unsupervised learning is to find the hidden patterns and useful insights from the unknown dataset.
Supervised learning needs supervision to train the model.	Unsupervised learning does not need any supervision to train the model.
Supervised learning can be categorized in Classification and Regression problems.	Unsupervised Learning can be classified in Clustering and Associations problems.
Supervised learning can be used for those cases where we know the input as well as corresponding outputs.	Unsupervised learning can be used for those cases where we have only input data and no corresponding output data.
Supervised learning model produces an accurate result.	Unsupervised learning model may give less accurate result as compared to supervised learning.
Supervised learning is not close to true Artificial intelligence as in this, we first train the model for each data, and then only it can predict the correct output.	Unsupervised learning is more close to the true Artificial Intelligence as it learns similarly as a child learns daily routine things by his experiences.
It includes various algorithms such as Linear Regression, Logistic Regression, Support Vector Machine, Multi-class Classification, Decision tree, Bayesian Logic, etc.	It includes various algorithms such as Clustering, KNN, and Apriori algorithm.

4.2 Hierarchical: AGNES, DIANA

(Written Notes From Page No – 15-27)

4.3 Partitional: K-means clustering, K-Mode Clustering

4.3.1 K-Mean Clustering

k-means clustering algorithm

One of the most used clustering algorithm is *k-means*. It allows to group the data according to the existing similarities among them in k clusters, given as input to the algorithm. I'll start with a simple example.

Let's imagine we have 5 objects (say 5 people) and for each of them we know two features (height and weight). We want to group them into $k=2$ clusters.

Our dataset will look like this:

	Height (H)	Weight (W)
Person 1	167	55
Person 2	120	32
Person 3	113	33
Person 4	175	76
Person 5	108	25

First of all, we have to initialize the value of the centroids for our clusters. For instance, let's choose Person 2 and Person 3 as the two centroids $c1$ and $c2$, so that $c1=(120,32)$ and $c2=(113,33)$.

Now we compute the euclidian distance between each of the two centroids and each point in the data. If you did all the calculations, you should have come up with the following numbers:

	Distance of object from $c1$	Distance of object from $c2$
Person 1	52.3	58.3
Person 2	0	7.1
Person 3	7.1	0
Person 4	70.4	75.4
Person 5	13.9	9.4

At this point, we will assign each object to the cluster it is closer to (that is taking the minimum between the two computed distances for each object).

We can then arrange the points as follows:

Person 1 → cluster 1
Person 2 → cluster 1
Person 3 → cluster 2

Person 4 → cluster 1
Person 5 → cluster 2

Let's iterate, which means to redefine the centroids by calculating the mean of the members of each of the two clusters.

So $c'1 = ((167+120+175)/3, (55+32+76)/3) = (154, 54.3)$ and $c'2 = ((113+108)/2, (33+25)/2) = (110.5, 29)$

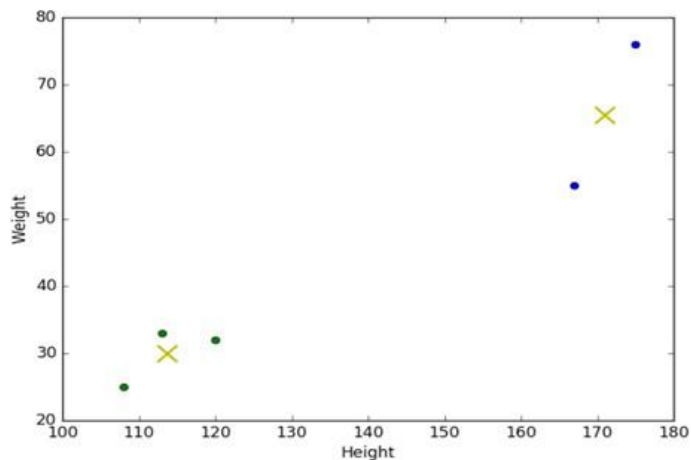
Then, we calculate the distances again and re-assign the points to the new centroids.

We repeat this process until the centroids don't move anymore (or the difference between them is under a certain small threshold).

In our case, the result we get is given in the figure below. You can see the two different clusters labelled with two different colours and the position of the centroids, given by the crosses.

How to apply k-means?

As you probably already know, I'm using Python libraries to analyze my data. The *k-means* algorithm is implemented in the *scikit-learn* package. To use it, you will just need the following line in your script:



What if our data is... non-numerical?

At this point, you will maybe have noticed something. The basic concept of *k-means* stands on mathematical calculations (means, euclidian distances). But what if our data is non-numerical or, in other words, *categorical*? Imagine, for instance, to have the ID code and date of birth of the five people of the previous example, instead of their heights and weights.

We could think of transforming our categorical values in numerical values and eventually apply *k-means*. But beware: *k-means* uses numerical distances, so it could consider close two really distant objects that merely have been assigned two close numbers.

4.3.2 K-Mode Clustering

Understanding K-Modes Clustering

K-Modes clustering is an extension of the K-Means algorithm tailored for categorical data. Unlike K-Means, which uses Euclidean distance, K-Modes employs a simple matching dissimilarity measure. The algorithm iteratively assigns data points to clusters based on the mode (most frequent category) of the cluster.

Key Concepts

1. **Dissimilarity Measure:** K-Modes uses the Hamming distance, which counts the number of mismatches between categorical attributes.
2. **Cluster Centroids:** Instead of mean values, K-Modes uses modes (most frequent categories) as cluster centroids.
3. **Cluster Assignment:** Data points are assigned to the cluster with the nearest mode.

Implementing K-Modes Clustering with Scikit-Learn

Scikit-Learn, a popular machine learning library in Python, provides a robust implementation of the K-Modes algorithm through the kmodes package. Let's walk through the steps to implement K-Modes clustering and reveal cluster features.

Step 1: Install Required Libraries

First, we ensure to have the necessary libraries installed. We can install the **kmodes** package using pip:

```
pip install kmodes
```

Step 2: Import Libraries and Load Data

Next, import the required libraries and load your categorical dataset. For this example, we'll use a sample dataset.

```
import pandas as pd
from kmodes.kmodes import KModes

data = {
    'Color': ['Red', 'Blue', 'Green', 'Blue', 'Red', 'Green', 'Red',
             'Blue'],
    'Shape': ['Circle', 'Square', 'Triangle', 'Circle', 'Square',
             'Triangle', 'Circle', 'Square'],
    'Size': ['Small', 'Large', 'Medium', 'Small', 'Large', 'Medium',
            'Small', 'Large']
}
df = pd.DataFrame(data)
```

Step 3: Apply K-Modes Clustering

Now, apply the K-Modes algorithm to cluster the data. We'll specify the number of clusters (k) and fit the model.

```
# Initialize K-Modes with 2 clusters
km = KModes(n_clusters=2, init='Huang', n_init=5, verbose=1)
clusters = km.fit_predict(df)
df['Cluster'] = clusters
```

Step 4: Reveal Cluster Features

To understand the characteristics of each cluster, we need to analyze the cluster centroids and the distribution of data points within each cluster.

```
# Cluster centroids
centroids = km.cluster_centroids_
print("Cluster Centroids:")
print(centroids)

# Cluster analysis
for cluster in range(km.n_clusters):
    print(f"\nCluster {cluster}:")
    cluster_data = df[df['Cluster'] == cluster]
    print(cluster_data.describe(include='all'))
```

Output:

```
Cluster Centroids:
[['Red' 'Circle' 'Small']
 ['Blue' 'Square' 'Large']]
```

```
Cluster 0:
   Color  Shape  Size  Cluster
0    Red  Circle  Small        0
4    Red  Square  Large        0
6    Red  Circle  Small        0
```

```
Cluster 1:
   Color  Shape  Size  Cluster
1   Blue  Square  Large        1
3   Blue  Circle  Small        1
7   Blue  Square  Large        1
```

Use-Cases and Applications of K-Modes Clustering

K-Modes clustering is particularly useful in various domains where categorical data is prevalent:

1. **Market Segmentation:** Grouping customers based on categorical attributes like preferences, buying behavior, and demographics.
2. **Healthcare:** Clustering patients based on categorical medical records to identify patterns in diseases and treatments.
3. **Social Media Analysis:** Categorizing users based on their activity, interests, and interactions.

4.3.4 Self-Organizing Map

A Self Organizing Map (SOM) or Kohonen Map is an unsupervised neural network algorithm based on biological neural models from the 1970s. It uses a competitive learning approach and is primarily designed for clustering and dimensionality reduction. SOM effectively maps high-dimensional data to a lower-dimensional grid enabling easier interpretation and visualization of complex datasets.

It consists of two primary layers:

- **Input Layer:** Represents the features of the data.
- **Output Layer:** Arranged as a 2D grid of neurons with each neuron representing a cluster in the data.

Working of Self Organizing Maps (SOM)

Here are the step by step explanation of its working:

1. Initialization

The weights of the output neurons are randomly initialized. These weights represent the features of each neuron and will be adjusted during training.

2. Competition

For each input vector, SOM computes the Euclidean distance between the input and the weight vectors of all neurons. The neuron with the smallest distance is the winning neuron.

$$\text{Formula: } D(j) = \sum_{i=1}^n (w_{ij} - x_i)^2$$

where

- $D(j)$ is the distance for neuron
- j and n is the number of features.

3. Weight Update

The winning neuron's weights are updated to move closer to the input vector. The weights of neighboring neurons are also adjusted but with smaller changes.

$$\text{Formula: } w_{ij}^{(new)} = w_{ij}^{(old)} + \alpha \cdot (x_i - w_{ij}^{(old)})$$

where

- α is the learning rate
- x_i is the input feature.

4. Learning Rate Decay

The learning rate α decreases over time allowing the map to converge to stable values.

$$\text{Formula: } \alpha(t + 1) = 0.5 \cdot \alpha(t)$$

5. Stopping Condition

The training stops when the maximum number of epochs is reached or when the weights converge.

4.5 Expectation Maximization(EM),

Written Notes Page No 13-14

4.6 Gaussian Mixture Models,

Written Notes Page No 11-13

Gaussian mixtures

* Linear Super-position of Gaussians

$$P(x) = \sum_{k=1}^K \pi_k N(x | \mu_k, \Sigma_k) \quad \text{--- ①}$$

π_k : Normal multi-variate Gaussian dist.
 Mixing coefficient: Weightage for each Gaussian dist.
 No. of Gaussians

* Normalization & positivity require:

$$0 \leq \pi_k \leq 1, \quad \sum_{k=1}^K \pi_k = 1$$

* consider log-likelihood:

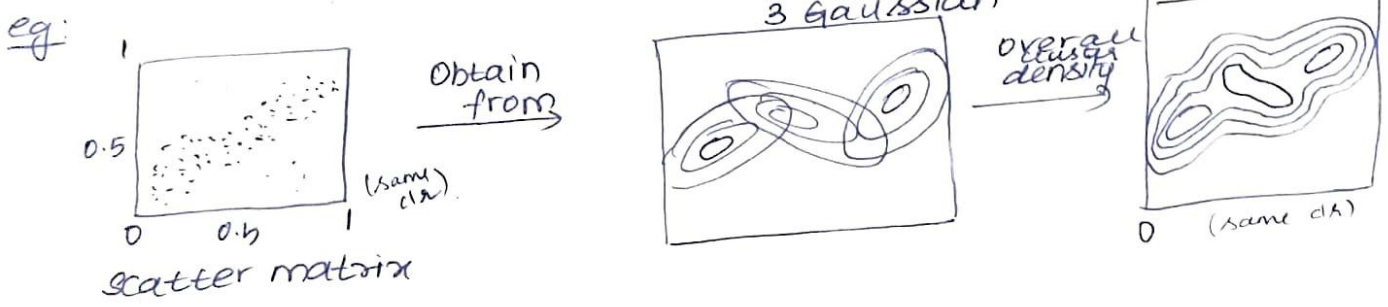
$$\ln P(x | \mu, \Sigma, \pi) = \sum_{n=1}^N \ln P(x_n) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k N(x_n | \mu_k, \Sigma_k) \right\} \quad \text{(from ①)}$$

no. of instance

• ML doesnot work here as there is no closed form soln

parameters can be calculated using -

" EXPECTATION MAXIMIZATION (EM) technique "



Latent variable : Posterior probability

- We can think of the mixing co-efficients as prior probabilities for the components
- For a gn. value of 'x', we can evaluate the corresponding posterior probabilities, called responsibility

Gaussian Mixture model (GMM)

The Gaussian distribution

* Univariate Gaussian Distribution

$$G(x|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

↓ ↓
mean variance

* Multivariate Gaussian Distribution

$$N(x|\mu, \Sigma) = \frac{1}{(2\pi|\Sigma|)^{1/2}} \exp\left\{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right\}$$

↓ ↓
mean co-variance

We need to estimate these parameters (Σ, μ) of a distribution:

* one method - Maximum Likelihood (ML) Estimation.

ML method for estimating parameters

* consider log of Gaussian distribution

$$\ln P(x|\mu, \Sigma) = -\frac{1}{2} \ln(2\pi) - \frac{1}{2} \ln|\Sigma| - \frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)$$

* Take the derivative & equate it to zero

$$\frac{\partial \ln P(x|\mu, \Sigma)}{\partial \mu} = 0 \qquad \frac{\partial \ln P(x|\mu, \Sigma)}{\partial \Sigma} = 0$$

$$\Downarrow$$
$$\mu_{ML} = \frac{1}{N} \sum_{n=1}^N x_n$$

$$\Sigma_{ML} = \frac{1}{N} \sum_{n=1}^N (x_n - \mu_{ML})(x_n - \mu_{ML})^T$$

where, $N \Rightarrow$ no. of samples or datapts.

What if we have mixture of Gaussians?

⇓
"Gaussian Mixture"

From Bayes rule,

$$\underset{\substack{\uparrow \\ \text{Latent} \\ \text{variable}}}{\gamma_k(x)} = P(k|x) = \frac{P(k) P(x|k)}{P(x)}$$
$$= \frac{\pi_k \mathcal{N}(x|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)} \quad \text{where, } \pi_k = \frac{N_k}{N}$$

No of samples for particular class
No. of samples

Interpret N_k as the effective no. of pts assigned to cluster k .

EXPECTATION MAXIMIZATION

→ EM alg. is an iterative optimization tech. which is operated locally

Estimation step:

for a gn. parameter values we can compute the expected values of the latent variable missing values.

Maximization step:

updates the parameters of our model based on the latent variable calculated using ML method.

EM alg. for GMM:

Gn. a GMM, the goal is to maximize the likelihood w.r.t parameters comprising the mean & co-variances of the components & the mixing coefficients.

1. Initialize the means μ_j , covariances Σ_j & mixing Co-eff. π_j & evaluate the initial value of log likelihood
(index)
2. E-Step: Evaluate the responsibilities using the current parameter values

$$\gamma_k(x) = \frac{\pi_k \mathcal{N}(x|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)}$$

3. M-Step: Re-estimate the parameters using the current responsibilities

$$\mu_j = \frac{\sum_{n=1}^N \gamma_j(x_n) x_n}{\sum_{n=1}^N \gamma_j(x_n)} \quad \Sigma_j = \frac{\sum_{n=1}^N \gamma_j(x_n) (x_n - \mu_j)(x_n - \mu_j)^T}{\sum_{n=1}^N \gamma_j(x_n)}$$

$$\pi_j = \frac{1}{N} \sum_{n=1}^N \gamma_j(x_n)$$

4. Evaluate log likelihood

$$\ln P(X|\mu, \Sigma, \pi) = \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k N(x_n | \mu_k, \Sigma_k) \right\}$$

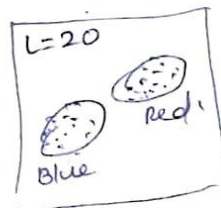
→ If there is no convergence, return to step 2.

↓
 $\mu, \Sigma, \pi \rightarrow$ no change in ^{successive} ~~prev~~ iterations.
 log likelihood $\rightarrow$ no change in ~~prev~~ iterations

EM alg: Example:

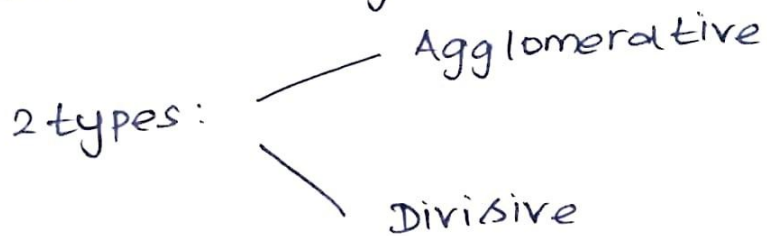


repeat
EM step
→
recompute



Hierarchical clustering

- In this approach, we partition the data by grouping it into a tree of clusters or a hierarchy
- Hierarchical representation of data is useful for data summarization & visualization.
- * Hierarchical clustering alg. combines and divides existing groups, creating a hierarchical str that showcases the order in which groups are divided or merged.



AGGLOMERATIVE HIERARCHICAL CLUSTERING METHOD:

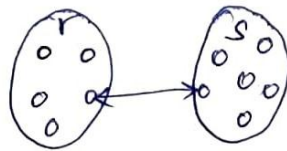
- Bottom-up approach is used
- Each object forms its own cluster.
- The process converges by iteratively merging clusters into larger clusters, until all the objects are in a single cluster or a termination condition is reached.
- The single cluster becomes the hierarchy root.
- During the merging operation, 2 similar clusters are identified using distance measures & combined to form a larger cluster.

- per iteration, two clusters are merged, where each cluster contains at least one object.
- An agglomerative method requires at the most 'n' iterations.

Three types:

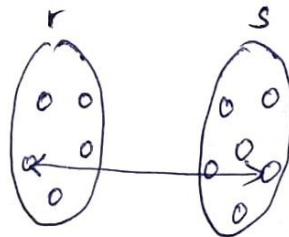
① Single link technique:

The dist. b/w two clusters is defined as the shortest dist. b/w 2 pts in each cluster.



② Complete link technique:

The dist. b/w two clusters is defined as the longest dist. b/w 2 pts in each cluster.



③ Average link technique:

The ^{avg} dist. b/w each pt. in one cluster to every pt in the other cluster.

Complete link:

P1 (1,1) P2 (1.5, 1.5) P3 (5,5) P4 (3,4) P5 (4,4)
P6 (3,3.5)

Step 1:

compute the dist. matrix by,

$$d((x,y)(a,b)) = \sqrt{(x-a)^2 + (y-b)^2}$$

	P1	P2	P3	P4	P5	P6
P1	0					
P2	0.71	0				
P3	5.66	4.95	0			
P4	3.6	2.92	2.24	0		
P5	4.24	3.53	1.41	1.0	0	
P6	3.20	2.5	2.5	0.5	1.12	0

Step 2:

Merging the 2 closest members of 2 clusters & find the min. element in dist. matrix.

$$\min = 0.5$$

Update the dist. matrix,

$$\max(d(P4, P6), P1) = \max(d(P4, P1), d(P6, P1)) = \max(3.6, 3.2) = 3.6$$

$$\max(d(P4, P6), P2) = \max(2.92, 2.5) = 2.92$$

$$\max(d(P4, P6), P3) = \max(2.24, 2.5) = 2.5$$

$$\max(d(P4, P6), P5) = \max(1.0, 1.12) = 1.12$$

	P1	P2	P3	P4, P6	P5
P1	0				
P2	0.71	0			
P3	5.66	4.95	0		
P4, P6	3.6	2.92	2.5	0	
P5	4.24	3.53	1.41	1.12	0

Now we combine P1 & P2

$$\max(d(P1, P2), P3) = \max(d(P1, P3), d(P2, P3)) \\ = \max(5.66, 4.95) = 5.66$$

$$\max(d(P1, P2), (P4, P6)) = \max(P1, P4, P6), (P2, P4, P6) \\ = \max(3.6, 2.9) = 3.6$$

$$\max(d(P1, P2), P5) = \max(4.24, 3.53) \\ = 4.24$$

	P1, P2	P3	P4, P6	P5
(P1, P2)	0			
P3	5.66	0		
(P4, P6)	3.6	2.5	0	
P5	4.24	1.41	1.12	0

Now we combine P4, P6 & P5

$$\max(d(P4, P6, P5), d(P1, P2)) = \max\{(P4, P6), (P1, P2), \\ (P5, P1, P2)\} \\ = \max(3.6, 4.24) = 4.24$$

$$\max(d(P4, P6, P5), P3) = \max(P4, P6, P3), (P5, P3) \\ = \max(2.5, 1.41) = 2.5$$

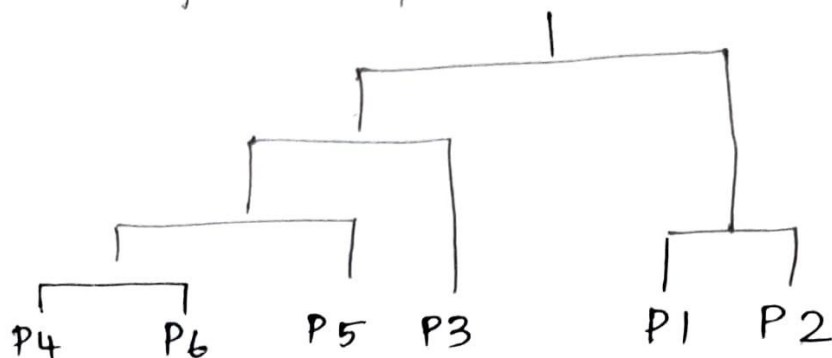
$\max(d(P$

	P1, P2	P3	P4, P6, P5
P1, P2	0		
P3	5.66	0	
P4, P6, P5	4.24	2.5	0

Now we combine P4, P6, P5 & P3.

$$\max(d(P4, P6, P5, P3), d(P1, P2)) = \max\{(P4, P6, P5), (P1, P2) \\ (P3, P1, P2)\} \\ = (4.24, 5.66) = 5.66$$

	(P_1, P_2)	P_4, P_6, P_5, P_3
P_1, P_2	0	
P_4, P_6, P_5, P_3	5.66	0



Average link:

* Same datapts. as above.

After cal. dist. matrix

	P_1	P_2	P_3	P_4	P_5	P_6
P_1	0					
P_2	0.71	0				
P_3	5.66	4.95	0			
P_4	3.6	2.92	2.24	0		
P_5	4.24	3.53	1.41	1.0	0	
P_6	3.20	2.5	2.5	0.5	1.12	0

so combine P_4, P_6

$$\begin{aligned} \text{avg}(d(P_4, P_6), d(P_2)) &= \text{avg}((P_4, P_6), (P_6, P_2)) \\ &= \text{avg}(\cancel{2.92}, \cancel{2.5}) \\ &= \text{avg}(3.6, 3.2) = 3.4 \end{aligned}$$

$$\text{avg}(d(P_4, P_6), d(P_3)) = \text{avg}(2.92, 2.5) = 2.71$$

$$\text{avg}(d(P_4, P_6), d(P_5)) = \text{avg}(2.24, 2.5) = 2.37$$

$$\text{avg}(d(P_4, P_6), d(P_5)) = \text{avg}(1.0, 1.12) = 1.06$$

Updated dist. matrix

	P1	P2	P3	P4, P6	P5
P1	0				
P2	0.71	0			
P3	5.66	4.95	0		
P4, P6	3.4	2.71	2.37	0	
P5	4.24	3.53	1.41	1.06	0

Now combine P1 & P2

$$\text{avg}(d(P1, P2), P3) = \text{avg}(d(P1, P3), d(P2, P3)) \\ = \text{avg}(5.66, 4.95) = 5.31$$

$$\text{avg}(d(P1, P2), (P4, P6)) = \text{avg}((P1, P4, P6) (P2, P4, P6)) \\ = \text{avg}(3.4, 2.71) = 2.96$$

$$\text{avg}(d(P1, P2), P5) = \text{avg}(P1, P5) (P2, P5) \\ = \text{avg}(4.24, 3.53) = 3.89$$

	P1, P2	P3	P4, P6	P5
P1, P2	0			
P3	5.31	0		
P4, P6	2.96	2.37	0	
P5	3.89	1.41	1.06	0

Now combine P4, P6 & P5

$$\text{avg}(d(P4, P6, P5) (P1, P2)) = \text{avg}(P4, P6, P1, P2) (P5, P1, P2) \\ = \text{avg}(2.96, 3.89) = 3.43$$

$$\text{avg}(d(P4, P6, P5) (P3)) = \text{avg}(P4, P6, P3) (P5, P3) \\ = \text{avg}(2.37, 1.41) = 1.96$$

(4)

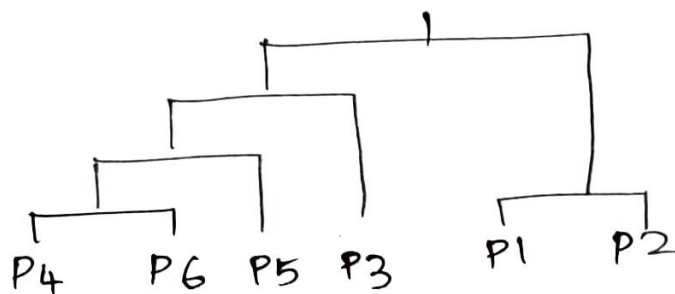
	P1, P2	P3	P4, P6, P5
(P1, P2)	0		
P3	5.31	0	
P4, P6, P5	3.43	1.96	0

Now combine P4, P6, P5 & P3

$$\text{avg}(d(P4, P6, P5, P3), (P1, P2)) = \text{avg}\{(P4, P6, P5)(P1, P2), (P3, P1, P2)\}$$

$$= \text{avg}(3.43, 5.31) = 4.37$$

	P1, P2	P4, P6, P5, P3
P1, P2	0	
P4, P6, P5, P3	4.37	0



Single link:

* Same datasets. as above

After cal. dist. matrix

	P1	P2	P3	P4	P5	P6
P1	0					
P2	0.71	0				
P3	5.66	4.95	0			
P4	3.6	2.92	2.24	0		
P5	4.24	3.53	1.41	1.0	0	
P6	3.2	2.5	2.5	0.5	1.12	0

Combine P4 & P6.

$$\min(d(P4, P6), P1) = \min\{(P4, P1), (P6, P1)\}$$

$$= \min(3.6, 3.2) = 3.2$$

$$\min(d(P4, P6), P2) = \min(2.92, 2.5) = 2.5$$

$$\min(d(P4, P6), P3) = \min(2.24, 2.5) = 2.24$$

$$\min(d(P4, P6), P5) = \min(1, 1.12) = 1$$

Update dist. matrix

	P1	P2	P3	P4, P6	P5
P1	0				
P2	0.71	0			
P3	5.66	4.95	0		
P4, P6	3.2	2.5	2.24	0	
P5	4.24	3.53	1.41	1	0

combine P_1 & P_2

$$\min(d(P_1, P_2), P_3) = \min\{(P_1, P_3), (P_2, P_3)\}$$

$$= \min(5.66, 4.95) = 4.95$$

$$\min(d(P_1, P_2), (P_4, P_6)) = \min(P_1, P_4, P_6), (P_2, P_4, P_6)$$

$$= \min(8.2, 2.5) = 2.5$$

$$\min(d(P_1, P_2), P_5) = \min(4.24, 3.53) = 3.53$$

update:

	P_1, P_2	P_3	P_4, P_6	P_5
P_1, P_2	0			
P_3	4.95	0		
P_4, P_6	2.5	2.24	0	
P_5	3.53	1.41	1	0

combine P_4, P_6 & P_5

$$\min(d(P_4, P_6, P_5), (P_1, P_2)) = \min\{(P_4, P_6, P_5), (P_5, P_1, P_2)\}$$

$$= \min(2.5, 3.53) = 2.5$$

$$\min(d(P_4, P_6, P_5), P_3) = \min\{(P_4, P_6, P_3), (P_5, P_3)\}$$

$$= \min(2.24, 1.41) = 1.41$$

update:

	P_1, P_2	P_3	P_4, P_6, P_5
P_1, P_2	0		
P_3	4.95	0	
P_4, P_6, P_5	2.5	1.41	0

(6)

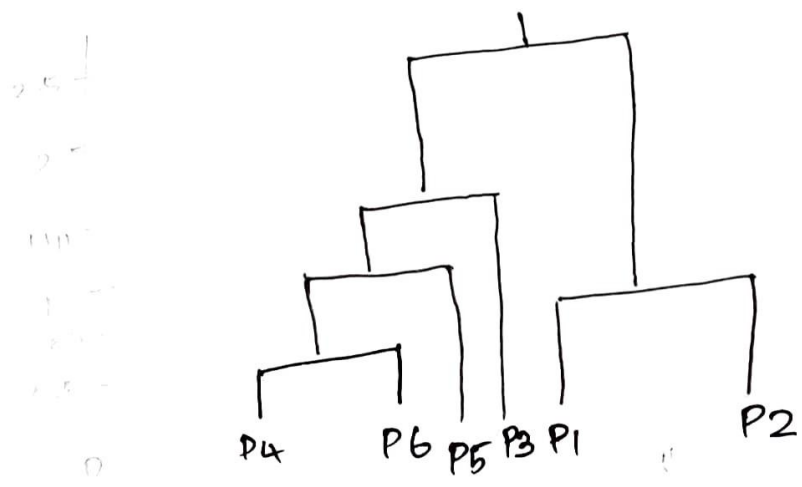
combine P_4, P_6, P_5 & P_3

$$\min(d(P_4, P_6, P_5, P_3), (P_1, P_2)) = \min\{(P_4, P_6, P_5)(P_1, P_2), (P_3, (P_1, P_2))\}$$
$$= \min(2.5, 4.95) = 2.5 //$$

update:

	P_1, P_2	P_4, P_6, P_5, P_3
P_1, P_2	0	
P_4, P_6, P_5, P_3	2.5	0

dendrogram:

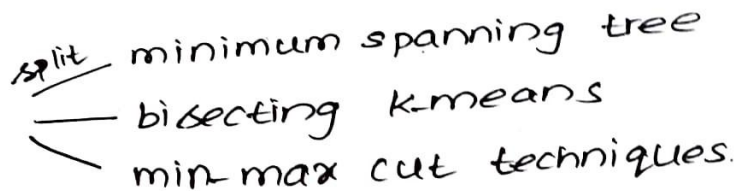


Divisive Hierarchical clustering

- Top-down approach
- recursively partition the cluster into small sub-clusters starting from root.
- We can stop the partitioning process when each cluster at the lowest level contains only one obj or the obj within cluster are sufficiently similar to each other.

PROCESS:

1. Start by placing all obj. in one cluster.
2. Repeat until all the clusters have single object
 - a) select a cluster with max inter-cluster distance to split
 - b) Replace the selected cluster with the sub clusters.

Divisive method 

- split minimum spanning tree
- bisecting k-means
- min-max cut techniques.

SPLITTING USING BISECTING K-means APPROACH

Process:

- ① Consider the set of points in a cluster & its centroid w .
- ② select a pt. at random C_L among the set of pts.
- ③ Construct the pt. C_R as the symmetric pt. of C_L w.r.t the centroid w such that,
$$\text{distance}(w, C_L) = \text{distance}(w, C_R)$$
- ④ separate the other pts. in the cluster into 2 groups : ones closest to C_R belong to

subcluster R, and the ones closest to C_L belong to the sub-cluster L.

⑤ Iterate steps 1 to 5 for the subclusters R & L.

Example:

Consider datapts.,

$a_1(2,7)$, $a_2(2,5)$, $a_3(3,6)$, $a_4(8,5)$, $a_5(7,4)$, $a_6(8,3)$

& split it into 2 clusters.

Soln:

① centroid w is computed as $(5,5)$ $\left[\begin{array}{l} \frac{2+2+3+8+7+8}{6} = 5 \\ \frac{7+5+6+5+4+3}{6} = 5 \end{array} \right]$

② Let $a_3(3,6)$ be assigned to C_L .

Based on symmetry of C_L to w , a_5 will be assigned to C_R .

③ Evaluating the dist, we see (a_1, a_3) , (a_2, a_3) , (a_4, a_5) , (a_6, a_5) , (a_1, a_5) , (a_2, a_5) , (a_4, a_3) , (a_6, a_3)

a_1 & a_2 are closer to a_3

a_4 & a_6 are closest to a_5

④ Hence $L = \{a_1, a_2, a_3\}$

$R = \{a_4, a_5, a_6\}$

> 2 clusters are created.

4.7 Principal Component Analysis

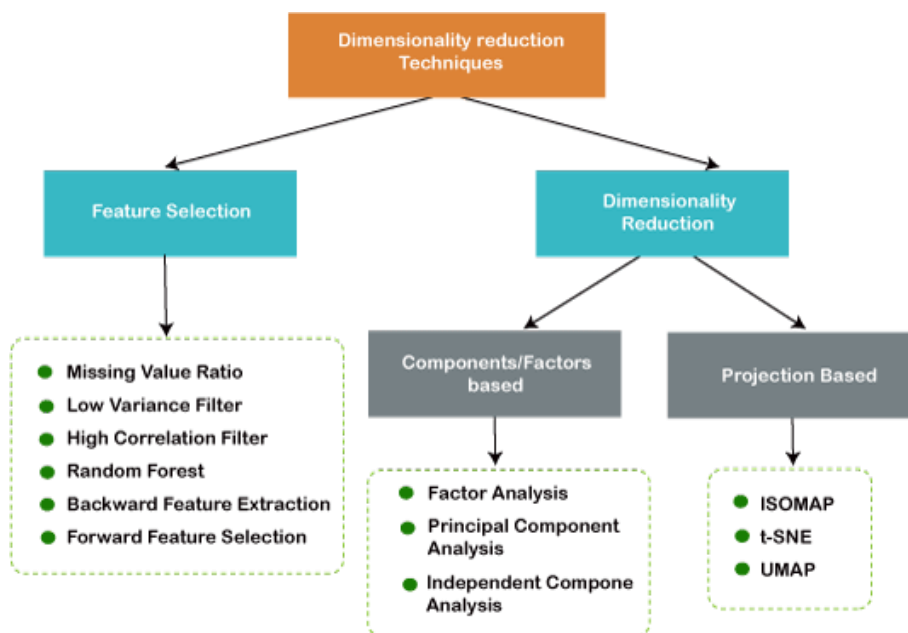
Dimensionality Reduction

The number of input features, variables, or columns present in a given dataset is known as dimensionality, and the process to reduce these features is called dimensionality reduction.

A dataset contains a huge number of input features in various cases, which makes the predictive modeling task more complicated. Because it is very difficult to visualize or make predictions for the training dataset with a high number of features, for such cases, dimensionality reduction techniques are required to use.

Dimensionality reduction technique can be defined as, "It is a way of converting the higher dimensions dataset into lesser dimensions dataset ensuring that it provides similar information." These techniques are widely used in machine learning for obtaining a better fit predictive model while solving the classification and regression problems.

It is commonly used in the fields that deal with high-dimensional data, such as speech recognition, signal processing, bioinformatics, etc. It can also be used for data visualization, noise reduction, cluster analysis, etc.



The Curse of Dimensionality

Handling the high-dimensional data is very difficult in practice, commonly known as the *curse of dimensionality*. If the dimensionality of the input dataset increases, any machine learning algorithm and model becomes more complex. As the number of features increases, the number of samples also gets increased proportionally, and the chance of overfitting also increases. If the machine learning model is trained on high-dimensional data, it becomes overfitted and results in poor performance.

Hence, it is often required to reduce the number of features, which can be done with dimensionality reduction.

Benefits of applying Dimensionality Reduction

Some benefits of applying dimensionality reduction technique to the given dataset are given below:

- By reducing the dimensions of the features, the space required to store the dataset also gets reduced.
- Less Computation training time is required for reduced dimensions of features.
- Reduced dimensions of features of the dataset help in visualizing the data quickly.
- It removes the redundant features (if present) by taking care of multicollinearity.

Disadvantages of dimensionality Reduction

There are also some disadvantages of applying the dimensionality reduction, which are given below:

- Some data may be lost due to dimensionality reduction.
- In the PCA dimensionality reduction technique, sometimes the principal components required to consider are unknown.

Approaches of Dimension Reduction

There are two ways to apply the dimension reduction technique, which are given below: Feature

Selection

Feature selection is the process of selecting the subset of the relevant features and leaving out the irrelevant features present in a dataset to build a model of high accuracy. In other words, it is a way of selecting the optimal features from the input dataset.

Three methods are used for the feature selection:

- **Filters Methods**

In this method, the dataset is filtered, and a subset that contains only the relevant features is taken. Some common techniques of filters method are:

- **Correlation**
- **Chi-Square Test**
- **ANOVA**
- **Information Gain, etc.**
- **Wrappers Methods**

The wrapper method has the same goal as the filter method, but it takes a machine learning model for its evaluation. In this method, some features are fed to the ML model, and evaluate the performance. The performance decides whether to add those features or remove to increase the accuracy of the model. This method is more accurate than the filtering method but complex to work. Some common techniques of wrapper methods are:

- Forward Selection
- Backward Selection
- Bi-directional Elimination
- **Embedded Methods:** Embedded methods check the different training iterations of the machine learning model and evaluate the importance of each feature. Some common techniques of Embedded methods are:
 - **LASSO**
 - **Elastic Net**
 - **Ridge Regression, etc.**

Feature Extraction:

Feature extraction is the process of transforming the space containing many dimensions into space with fewer dimensions. This approach is useful when we want to keep the whole information but use fewer resources while processing the information.

Some common feature extraction techniques are:

- a. Principal Component Analysis
- b. Linear Discriminant Analysis
- c. Kernel PCA
- d. Quadratic Discriminant Analysis

Common techniques of Dimensionality Reduction

- a. **Principal Component Analysis**
- b. **Backward Elimination**
- c. **Forward Selection**
- d. **Score comparison**
- e. **Missing Value Ratio**
- f. **Low Variance Filter**
- g. **High Correlation Filter**
- h. **Random Forest**
- i. **Factor Analysis**
- j. **Auto-Encoder**

Principal Component Analysis

Principal Component Analysis is an unsupervised learning algorithm that is used for the dimensionality reduction in [machine learning](#). It is a statistical process that converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation. These new transformed features are called the **Principal Components**. It is one of the popular tools that is used for exploratory data analysis and predictive modeling. It is a technique to draw strong patterns from the given dataset by reducing the variances.

PCA generally tries to find the lower-dimensional surface to project the high-dimensional data.

PCA works by considering the variance of each attribute because the high attribute shows the good split between the classes, and hence it reduces the dimensionality. Some real-world applications of PCA are *image processing*, *movie recommendation system*, *optimizing the power allocation in various communication channels*. It is a feature extraction technique, so it contains the important variables and drops the least important variable.

The PCA algorithm is based on some mathematical concepts such as:

- Variance and Covariance
- Eigenvalues and Eigen factors

Some common terms used in PCA algorithm:

- **Dimensionality:** It is the number of features or variables present in the given dataset. More easily, it is the number of columns present in the dataset.
- **Correlation:** It signifies that how strongly two variables are related to each other. Such as if one changes, the other variable also gets changed. The correlation value ranges from -1 to +1. Here, -1 occurs if variables are inversely proportional to each other, and +1 indicates that variables are directly proportional to each other.
- **Orthogonal:** It defines that variables are not correlated to each other, and hence the correlation between the pair of variables is zero.
- **Eigenvectors:** If there is a square matrix M , and a non-zero vector v is given. Then v will be eigenvector if Av is the scalar multiple of v .
- **Covariance Matrix:** A matrix containing the covariance between the pair of variables is called the Covariance Matrix.

Principal Components in PCA

As described above, the transformed new features or the output of PCA are the Principal Components. The number of these PCs are either equal to or less than the original features present in the dataset. Some properties of these principal components are given below:

- The principal component must be the linear combination of the original features.
- These components are orthogonal, i.e., the correlation between a pair of variables is zero.

- The importance of each component decreases when going to 1 to n, it means the 1 PC has the most importance, and n PC will have the least importance.

Steps for PCA algorithm

1. **Getting the dataset**

Firstly, we need to take the input dataset and divide it into two subparts X and Y, where X is the training set, and Y is the validation set.

2. **Representing data into a structure**

Now we will represent our dataset into a structure. Such as we will represent the two-dimensional matrix of independent variable X. Here each row corresponds to the data items, and the column corresponds to the Features. The number of columns is the dimensions of the dataset.

3. **Standardizing the data**

In this step, we will standardize our dataset. Such as in a particular column, the features with high variance are more important compared to the features with lower variance.

If the importance of features is independent of the variance of the feature, then we will divide each data item in a column with the standard deviation of the column. Here we will name the matrix as Z.

4. **Calculating the Covariance of Z**

To calculate the covariance of Z, we will take the matrix Z, and will transpose it. After transpose, we will multiply it by Z. The output matrix will be the Covariance matrix of Z.

5. **Calculating the Eigen Values and Eigen Vectors**

Now we need to calculate the eigenvalues and eigenvectors for the resultant covariance matrix Z. Eigenvectors or the covariance matrix are the directions of the axes with high information. And the coefficients of these eigenvectors are defined as the eigenvalues.

6. **Sorting the Eigen Vectors**

In this step, we will take all the eigenvalues and will sort them in decreasing order, which means from largest to smallest. And simultaneously sort the eigenvectors accordingly in matrix P of eigenvalues. The resultant matrix will be named as P*.

7. **Calculating the new features Or Principal Components**

Here we will calculate the new features. To do this, we will multiply the P* matrix to the Z. In the resultant matrix Z*, each observation is the linear combination of original features. Each column of the Z* matrix is independent of each other.

8. **Remove less or unimportant features from the new dataset.**

The new feature set has occurred, so we will decide here what to keep and what to remove. It means, we will only keep the relevant or important features in the new dataset, and unimportant features will be removed out.

Example:

Let's suppose that our data set is 2-dimensional with 2 variables x, y and that the eigenvectors and eigenvalues of the covariance matrix are as follows:

$$v1 = \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix} \quad \lambda_1 = 1.284028$$

$$v2 = \begin{bmatrix} -0.7351785 \\ 0.6778736 \end{bmatrix} \quad \lambda_2 = 0.04908323$$

If we rank the eigenvalues in descending order, we get $\lambda_1 > \lambda_2$, which means that the eigenvector that corresponds to the first principal component (PC1) is $v1$ and the one that corresponds to the second component (PC2) is $v2$.

After having the principal components, to compute the percentage of variance (information) accounted for by each component, we divide the eigenvalue of each component by the sum of eigenvalues. If we apply this on the example above, we find that PC1 and PC2 carry respectively 96% and 4% of the variance of the data.

Applications of Principal Component Analysis

- PCA is mainly used as the dimensionality reduction technique in various AI applications such as **computer vision, image compression, etc.**
- It can also be used for finding hidden patterns if data has high dimensions. Some fields where PCA is used are Finance, data mining, Psychology, etc.

Latent Dirichlet Allocation

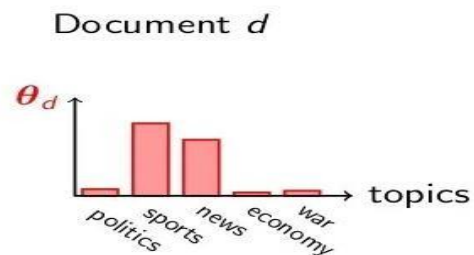
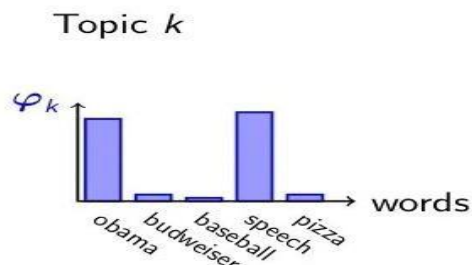
Latent Dirichlet Allocation (LDA) is used as a topic modelling technique that can classify text in a document to a particular topic. It uses Dirichlet distribution to find topics for each document model and words for each topic model.

Johann Peter Gustav Lejeune Dirichlet was a German mathematician in the 1800s who contributed widely to the field of modern mathematics. There is a probability distribution named after him '**Dirichlet Distribution**' which is the basis of LDA.

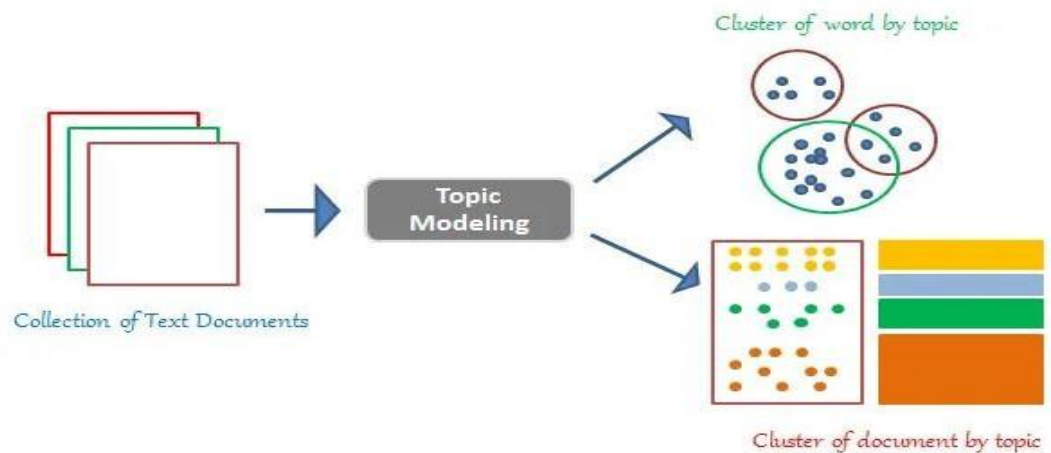
Latent Dirichlet Allocation

LDA discovers topics into a collection of documents.

LDA tags each document with topics.



How LDA works?



There are 2 parts in LDA:

- The words that belong to a document, that we already know.
- The words that belong to a topic or the probability of words belonging into a topic, that we need to calculate.

THE ALGORITHM TO FIND THE LATTER

- Go through each document and randomly assign each word in the document to one of k topics (k is chosen beforehand).
- For each document d , go through each word w and compute :
 1. $p(\text{topic } t \mid \text{document } d)$: the proportion of words in document d that are assigned to topic t . Tries to capture how many words belong to the topic t for a given document d . Excluding the current word.
If a lot of words from d belongs to t , it is more probable that word w belongs to t .
($\frac{\text{\#words in } d \text{ with } t + \alpha}{\text{\#words in } d \text{ with any topic} + k * \alpha}$)
 2. $p(\text{word } w \mid \text{topic } t)$: the proportion of assignments to topic t over all documents that come from this word w . Tries to capture how many documents are in topic t because of word w .

LDA represents documents as a mixture of topics. Similarly, a topic is a mixture of words. If a word has high probability of being in a topic, all the documents having w will be more strongly associated with t as well. Similarly, if w is not very probable to be in t , the documents which contain the w will be having very low probability of being in t , because rest of the words in d will belong to some other topic and hence d will have a higher probability for those topic. So even if w gets added to t , it won't be bringing many such documents to t .

- Update the probability for the word w belonging to topic t , as

$$p(\text{word } w \text{ with topic } t) = p(\text{topic } t \mid \text{document } d) * p(\text{word } w \mid \text{topic } t)$$

Assumptions of LDA for Topic Modelling:

Documents with similar topics use similar groups of words

Latent topics can then be found by searching for groups of words that frequently occur together in documents across the corpus

Documents are probability distributions over latent topics which signifies certain document will contain more words of a specific topic.

5 sentences, 2 topics:

- I like to eat broccoli and bananas.
 - I ate a banana and spinach smoothie for breakfast.
 - Chinchillas and kittens are cute.
 - My sister adopted a kitten yesterday.
 - Look at this cute hamster munching on a piece of broccoli.
Then he does some "calculations"
 - Sentences 1 and 2: 100% Topic A
 - Sentences 3 and 4: 100% Topic B
 - Sentence 5: 60% Topic A, 40% Topic B
- And take guesses of the topics:
- Topic A: 30% broccoli, 15% bananas, 10% breakfast, 10% munching, ...
at which point, you could interpret topic A to be about food
 - Topic B: 20% chinchillas, 20% kittens, 20% cute, 15% hamster, ...
at which point, you could interpret topic B to be about cute animals

Your question is how did he come up with those numbers? Which words in these sentences carry "information":

- broccoli, bananas, smoothie, breakfast, munching, eat
- chinchilla, kitten, cute, adopted, hamster

Now let's go sentence by sentence getting words from each topic:

- food 3, cute 0 --> food
- food 5, cute 0 --> food
- food 0, cute 3 --> cute
- food 0, cute 2 --> cute
- food 2, cute 2 --> 50% food + 50% cute

So my numbers, differ slightly from Chen's. Maybe he includes the word "piece" in "piece of broccoli" as counting towards food.

We made two calculations in our heads:

- to look at the sentences and come up with 2 topics in the first place. LDA does this by considering each sentence as a "mixture" of topics and guessing the parameters of each topic.

- to decide which words are important. LDA uses "term-frequency/inverse-document-frequency" to understand this.

LDA Procedure

Step1: Go through each document and randomly assign each word in the document to one of K topics (K is chosen beforehand)

Step2: This random assignment gives topic representations of all documents and word distributions of all the topics, albeit not very good ones

So, to improve upon them: For each document d, go through each word w and compute:

- $p(\text{topic } t \mid \text{document } d)$: proportion of words in document d that are assigned to topic t
- $p(\text{word } w \mid \text{topic } t)$: proportion of assignments to topic t, over all documents d, that come from word w

Step3: Reassign word w a new topic t' , where we choose topic t' with probability

- $p(\text{topic } t' \mid \text{document } d) * p(\text{word } w \mid \text{topic } t')$

This generative model predicts the probability that topic t' generated word w. we will iterate this last step multiple times for each document in the corpus to get steady-state.

Solved calculation

Let's say you have two documents.

Doc i: **"The bank called about the money."**

Doc ii: **"The bank said the money was approved."**

After removing the stop words, capitalization, and punctuation.

Unique words in corpus: **bank called about money boat approved**

Randomly assign topics

	1	2	2	1
Doc i	bank	called	about	money

K=2 (two topics) in our case

Similarly, done to each document in the corpus

	2	1	1	2
Doc ii	bank	said	money	approved

Next then,

Maintain the global statistics

word\topic	Topic 1	Topic 2
bank	1	1
called	0	1
about	0	1
money	2	0
said	1	0
approved	0	1

1	2	2	1
bank	called	about	money

Doc i

	Topic 1	Topic 2
Doc i	2	2

Total counts from all docs

After then, we will randomly select a word from doc i (word **bank** with topic assignment 1) and we will remove its assigned topic and we will calculate the probability for its new assignment.

Randomly re-assign topics

word\topic	Topic 1	Topic 2
bank	0 1	1
called	0	1
about	0	1
money	2	0
said	1	0
approved	0	1

1	2	2	1
bank	called	about	money

	Topic 1	Topic 2
Doc i	1 2	2

decrement the count

Probability of new assignment

$$P(\text{topic } t \mid \text{doc } d) = \frac{\text{Word in doc}(n) \text{ with topic}(K) + \alpha}{\text{Word in doc}(n) \text{ with topics} + \text{Number of topics} * \alpha}$$

How much doc likes topic

Smoothing parameter from Dirichlet

$$P(\text{word } w \mid \text{topic } t) = \frac{\text{Word (bank) with topic (K)} + \beta}{\text{Word with topic}(K) + \text{Unique word}(V) * \beta}$$

How much topic likes word

Size of vocab

For the
topic k=1

Probability of new assignment

word\topic	Topic 1	Topic 2
bank	0	1
called	0	1
about	0	1
money	2	0
said	1	0
approved	0	1

?	2	2	1
bank	called	about	money

Doc i

	Topic 1	Topic 2
Doc i	1	2

Our hyperparameters are:

- alpha = 0.5
- beta = 0.01
- 'topics' = 2 (i.e K= 1,2)
- 'iterations' = 1.

$$P_{11} = \frac{\text{Word in doc}(n=i) \text{ with topic}(K=1) + \alpha}{\text{Word in doc}(n=i) \text{ with topics} + \text{Number of topics} * \alpha} = \frac{1+0.5}{3+2*0.5} = 0.375$$

$$P_{12} = \frac{\text{Word (bank) with topic (K=1)} + \beta}{\text{Word with topic}(K=1) + \text{Unique word}(V) * \beta} = \frac{0+0.01}{3+6*0.01} = 0.003268$$

For the

Probability of new assignment

word\topic	Topic 1	Topic 2
bank	0	1
called	0	1
about	0	1
money	2	0
said	1	0
approved	0	1

?	2	2	1
bank	called	about	money

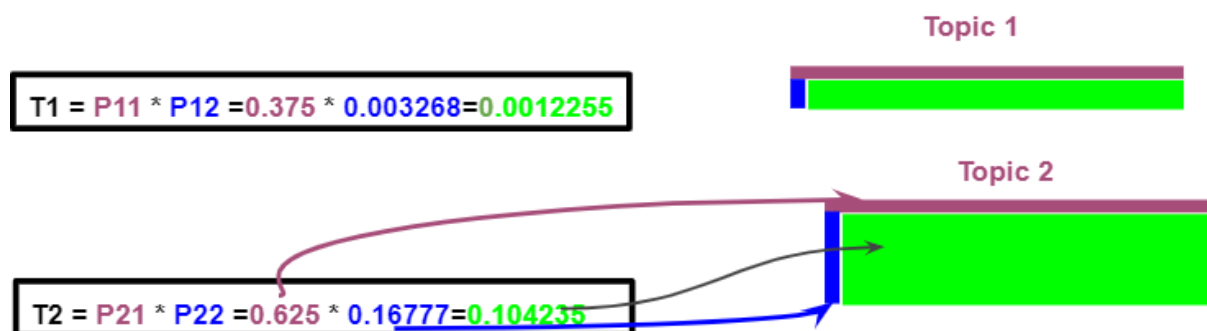
	Topic 1	Topic 2
Doc i	1	2

$$\begin{aligned}
 P_{21} &= \frac{\text{Word in doc}(n=i) \text{ with topic}(K=2) + \alpha}{\text{Word in doc}(n=i) \text{ with topics} + \text{Number of topics} * \alpha} = \frac{2+0.5}{3+2*0.5} = 0.625 \\
 P_{22} &= \frac{\text{Word (bank) with topic (K=2) + beta}}{\text{Word with topic}(K=2) + \text{Unique word}(V) * \beta} = \frac{1+0.01}{4+6*0.01} = 0.16777
 \end{aligned}$$

topic k=2

Now we will calculate the product of those two probabilities as given below:

Probability of new assignment



Good fit for both **document** and **word** for topic 2 (area is **greater**) than topic 1. So, our new assignment for word **bank** will be topic 2.

Now, we will update the count due to new assignment.

Update counts

word\topic	Topic 1	Topic 2
bank	0	1 2
called	0	1
about	0	1
money	2	0
said	1	0
approved	0	1

2	2	2	1	1
bank	called	about	money	boat

	Topic 1	Topic 2
Doc i	1	3 2

Increment the count based on new assignment

Now we will repeat the same step of reassignment. and iterate through each word of the whole

corpus.

Iterate through all words/docs

2	?	2	1	1
bank	called	about	money	boat

word\topic	Topic 1	Topic 2
bank	1	1
called	0	1 0
about	0	1
money	2	0
said	1	0
approved	0	1

	Topic 1	Topic 2
Doc i	1	2 3

decrement the count

eg. Use PCA to reduce dimension from 2 to 1

feature	E1	E2	E3	E4
x	4	8	13	7
y	11	4	5	14

Step 1:

No. of features $n = 2$

No. of samples $N = 4$

Step 2:

compute mean of variables

$$\bar{x} = (4 + 8 + 13 + 7) / 4 \Rightarrow 8$$

$$\bar{y} = (11 + 4 + 5 + 14) / 4 \Rightarrow 8.5$$

Step 3:

computation co-var. Matrix

ordered pairs: (x, x) (x, y) (y, x) (y, y)

① co-var of all ordered pairs

$$\text{cov}(x, x) = \frac{1}{N-1} \sum_{k=1}^N (x_{ik} - \bar{x}_i)(x_{jk} - \bar{x}_j)$$

$$= \frac{1}{3} [(4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2]$$

$$= 14.1$$

$$\text{cov}(x, y) = \frac{1}{3} \left[(4-8)(11-8.5) + (8-8)(4-8.5) + (13-8)(5-8.5) + (7-8)(14-8.5) \right]$$

$$= -11$$

$$\text{cov}(y, x) = \text{cov}(x, y) = -11$$

$$\text{cov}(y, y) = \frac{1}{3} \left((11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (14-8.5)^2 \right)$$

$$= 23$$

② Co-var matrix of size $n \times n$ i.e. 2×2

$$S = \begin{bmatrix} \text{cov}(x, x) & \text{cov}(x, y) \\ \text{cov}(y, x) & \text{cov}(y, y) \end{bmatrix}$$

$$= \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Step 4:

construct eigen value, eigen vector

Normalized eigen vector

(1) Eigen value:

$$\det(S - \lambda I) = 0$$

eigen value identity matrix

$$\det \begin{bmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{bmatrix} = 0$$

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\downarrow$$

$$I \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\downarrow$$

$$\begin{pmatrix} \lambda & 0 \\ 0 & \lambda \end{pmatrix}$$

$$(14-\lambda)(23-\lambda) - (-11)(-11) = 0$$

$$\lambda^2 - 37\lambda + 201 = 0 \Rightarrow \frac{1}{2a} \sqrt{b^2 - 4ac}$$

$\downarrow$ $\downarrow$ $\downarrow$
 -37 1 201

$$\lambda = 30.3849, 6.6151$$

(λ_1) (λ_2)

$$\lambda_1 > \lambda_2$$

largest eigen value $\rightarrow$ PCA
 $\nwarrow$
 λ_1

(ii) Eigen vector of λ_1

$$(S - \lambda_1 I) u_1 = 0$$

$$\begin{bmatrix} 14-\lambda_1 & -11 \\ -11 & 23-\lambda_1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = 0$$

$$\begin{bmatrix} (14-\lambda_1)u_1 - 11u_2 \\ -11u_1 + (23-\lambda_1)u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$(14-\lambda_1)u_1 - 11u_2 = 0$$

$$-11u_1 + (23-\lambda_1)u_2 = 0$$

$$\frac{u_1}{11} = \frac{u_2}{14-\lambda_1} = 1$$

when $t=1$, $u_1=11$, $u_2=14-\lambda_1$

eigen vector u_1 of $\lambda_1 = \begin{bmatrix} 11 \\ 14-\lambda_1 \end{bmatrix}$

$$= \begin{bmatrix} 11 \\ 14-30.3849 \end{bmatrix} = \begin{bmatrix} 11 \\ -16.3849 \end{bmatrix}$$

(iii) Normalize eigen vector u_1

$$e_1 = \frac{\begin{bmatrix} 11 \\ -16.3849 \end{bmatrix}}{\sqrt{11^2 + (-16.38)^2}}$$

$$= \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

$$e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}$$

Step 5: Derive new dataset

First PC	E_1 P_{11}	E_2 P_{12}	E_3 P_{13}	E_4 P_{14}
----------	-------------------	-------------------	-------------------	-------------------

$$P_{11} = e_1^T \begin{bmatrix} 4-8 \\ 11-8.5 \end{bmatrix} = [0.5574 \quad -0.8303] \cdot \begin{bmatrix} -4 \\ 2.5 \end{bmatrix}$$

$$P_{12} = [0.5574 \quad -0.8303] \begin{bmatrix} 8-8 \\ 4-8.5 \end{bmatrix} = 3.7361$$

$$P_{13} = 5.6928$$

$$P_{14} = -5.1238$$

Result: PC1

$$\begin{bmatrix} -4.3052 & 3.7361 & 5.6928 & -5.1238 \end{bmatrix}$$

